

گزارش کار پروژه دوم آزمایشگاه سیستم عامل

فراخوانی سیستمی، مولد شبه تصادفی، اطلاعات پردازش و مرتب سازی در xv6

هدف پروژه دوم، آشنایی عملی با مسیر اجرای system call در xv6 و اضافه کردن چند فراخوانی سیستمی جدید به هسته بود. در xv6 برنامه های user-space برای انجام کارهای حساس مانند دسترسی به فایل سیستم، دریافت اطلاعات پردازشها یا استفاده از داده های داخلی هسته، مستقیماً به ساختارهای kernel دسترسی ندارند و باید از system call استفاده کنند.

نویسندگان

معراج رستگار - 810102576

علی صادقی - 810102471

معراج پورحسینی - 810102420

- ۱ خلاصه اجرایی
- ۲ مسیر کلی اضافه کردن system call در xv6
- ۳ پاسخ پرسش‌های تئوری system call
 - ۳-۱ پرسش ۱. نقش کتابخانه‌های سطح کاربر و wrapperها
 - ۳-۲ پرسش ۲. روش‌های دیگر ارتباط user space با kernel در لینوکس
 - ۳-۳ پرسش ۳. چرا فقط syscall با DPL_USER فعال است؟
 - ۳-۴ پرسش ۴. چرا ss و esp فقط هنگام تغییر سطح دسترسی push می‌شوند؟
 - ۳-۵ پرسش ۵. نقش argptr و argint
- ۴ بررسی syscall با GDB
 - ۴-۱ کاربرد bt
 - ۴-۲ کاربرد down و بررسی eax
- ۵ بخش اول عملی: مولد عدد شبه‌تصادفی
 - ۵-۱ طراحی syscall setSeed
 - ۵-۲ طراحی syscall getRandomNumber
 - ۵-۳ برنامه تست dice
 - ۵-۴ پرسش‌های بخش random
 - ۵-۴-۱ ۱. چرا مولد عدد تصادفی در kernel پیاده‌سازی می‌شود؟
 - ۵-۴-۲ ۲. اگر دو بار پشت سر هم setSeed اجرا شود، خروجی تغییر می‌کند؟
- ۶ بخش دوم عملی: اطلاعات پردازش
 - ۶-۱ طراحی syscall process_information
 - ۶-۲ محاسبه parent، children، siblings و depth
 - ۶-۳ برنامه تست pinfo_test
 - ۶-۴ پرسش‌های بخش پردازش
 - ۶-۴-۱ ۱. xv6 های پردازش در state ۱.
 - ۶-۴-۲ ۲. اگر پردازش pid خودش را بدهد، state چه می‌شود؟
 - ۶-۴-۳ ۳. آیا می‌توان process_information را در user-space پیاده‌سازی کرد؟
- ۷ بخش سوم عملی: مرتب‌سازی اعداد
 - ۷-۱ طراحی syscall sort_numbers
 - ۷-۲ کار با فایل در kernel
 - ۷-۳ الگوریتم مرتب‌سازی

۷-۴ برنامه‌های تست sort

۷-۵ مقایسه mode switch ها

۷-۶ پرسش‌های بخش sort

۷-۶-۱ ۱. تعداد تغییر mode ها در هر بخش

۷-۶-۲ ۲. آیا لازم بود sort در kernel پیاده‌سازی شود؟

۷-۶-۳ ۳. دستور sort در Linux کجاست؟

۷-۶-۴ ۴. بخش kernel مسئول چه کارهایی است؟

۷-۶-۵ ۵. چرا نباید از الگوریتم مرتب‌سازی بازگشتی استفاده کنیم؟

۸ راهنمای build، اجرا و برنامه‌های اضافه‌شده

۹ اعتبارسنجی نهایی و سلامت تحویل

۱۰ جمع‌بندی

هدف پروژه دوم، آشنایی عملی با مسیر اجرای system call در xv6 و اضافه کردن چند فراخوانی سیستمی جدید به هسته بود. در برنامه‌های user-space برای انجام کارهای حساس مانند دسترسی به فایل سیستم، دریافت اطلاعات پردازنده یا استفاده از داده‌های داخلی هسته، مستقیماً به ساختارهای kernel دسترسی ندارند و باید از system call استفاده کنند.

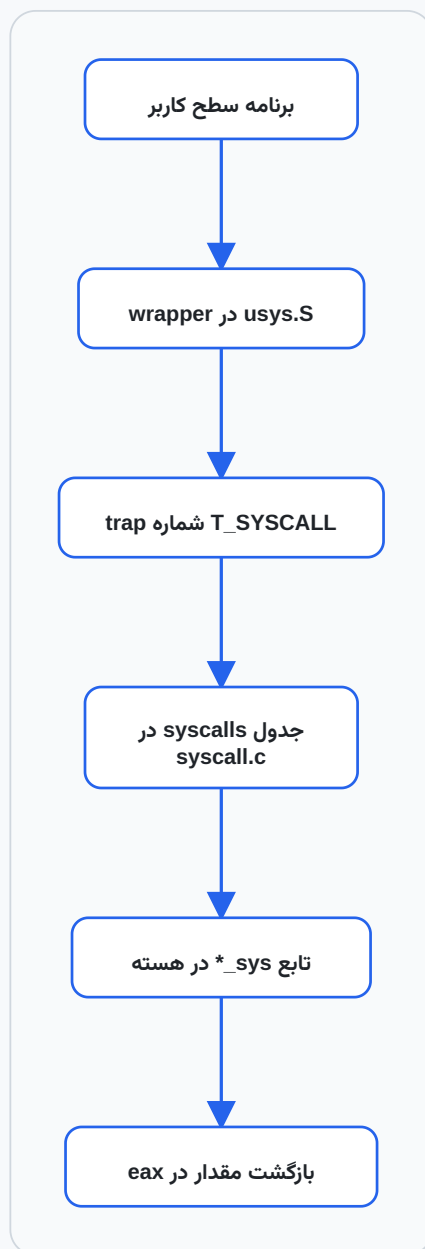
در این پروژه ابتدا مسیر فراخوانی syscall از `usys.S` تا `syscall.c` و سپس تابع‌های `sys_*` بررسی شد. بعد سه بخش عملی پیاده‌سازی شد:

1. مولد عدد شبه‌تصادفی در هسته با syscalls `setSeed` و `getRandomNumber`؛
2. نمایش اطلاعات پردازنده با `process_information` syscall؛
3. مرتب‌سازی اعداد داخل هسته با `sort_numbers` syscall و مقایسه آن با پیاده‌سازی `user-space`.

نکته

در این پروژه هر فراخوانی سیستمی از دو طرف بررسی شده است: از سمت برنامه کاربر که تابعی معمولی را صدا می‌زند و از سمت هسته که آرگومان‌ها را بررسی می‌کند و عملیات اصلی را انجام می‌دهد.

نمودار زیر مسیر کلی ورود یک برنامه کاربر به هسته را برای فراخوانی‌های سیستمی این پروژه خلاصه می‌کند:



رابطه ساده‌ای که برای مولد شبه‌تصادفی استفاده می‌شود از نوع خطی هم‌نهشتی است و به صورت زیر در نظر گرفته می‌شود:

$$X_{n+1} = (aX_n + c) \bmod m$$

مسیر کلی اضافه کردن system call در xv6

برای اضافه کردن هر system call جدید، چند فایل باید با هم هماهنگ شوند. در این پروژه همین مسیر برای چهار syscall جدید انجام شد:

فایل	کاری که انجام شد
syscall.h	تعریف شماره syscall های جدید
syscall.c	معرفی prototype تابع های <code>sys_*</code> و اضافه کردن آن ها به آرایه <code>syscalls[]</code>
sysproc.c	پیاده سازی syscall های مربوط به پردازش و random
sysfile.c	پیاده سازی syscall مربوط به sort فایل
user.h	اضافه کردن prototype قابل استفاده در برنامه های سطح کاربر
usys.S	ساخت wrapper اسمبلی با ماکروی SYSCALL
Makefile	اضافه کردن برنامه های تست به UPROGS

شماره های اضافه شده در پروژه به این صورت هستند:

syscall.h

```
1  #define SYS_setSeed          22
2  #define SYS_getRandomNumber  23
3  #define SYS_process_information 24
4  #define SYS_sort_numbers     25
```

در `usys.S` نیز برای هر کدام wrapper ساخته شد:

usys.S

```
1  SYSCALL(setSeed)
2  SYSCALL(getRandomNumber)
3  SYSCALL(process_information)
4  SYSCALL(sort_numbers)
```

بنابراین برنامه `user` بدون دانستن جزئیات `trap` و شماره syscall فقط تابعی مثل `setSeed()` را صدا می زند.

پاسخ پرسش‌های تئوری system call

پرسش ۱. نقش کتابخانه‌های سطح کاربر و wrapperها

در برنامه xv6، user مستقیماً دستور `int` را نمی‌نویسد. در فایل `usys.S` یک ماکرو به نام `SYSCALL` وجود دارد که برای هر `syscall` یک تابع `wrapper` می‌سازد. این `wrapper` شماره `syscall` را در ثبات `eax` قرار می‌دهد و سپس `trap` مربوط به `syscall` را اجرا می‌کند:

wrapper فراخوانی سیستمی

```
1  movl $SYS_name, %eax
2  int $T_SYSCALL
3  ret
```

از طرف دیگر، `user.h` prototype تابع‌ها را در اختیار برنامه‌نویس می‌گذارد. بنابراین برنامه‌نویس به جای دانستن شماره `syscall`، شکل پشته، `trap number` و `ABI`، فقط تابعی مثل `getpid()` یا `sort_numbers(path)` را صدا می‌زند.

این انتزاع سه فایده دارد:

1. **سادگی برنامه‌نویسی:** کاربر با تابع معمولی C کار می‌کند.
2. **قابلیت حمل بهتر:** اگر شماره یا روش فراخوانی `syscall` تغییر کند، برنامه `user` تغییر زیادی نمی‌خواهد.
3. **امنیت و کنترل:** ورود به `kernel` فقط از مسیرهای مشخص و کنترل‌شده انجام می‌شود.

پرسش ۲. روش‌های دیگر ارتباط user space با kernel در لینوکس

در لینوکس نیز `system call` پایه اصلی ارتباط با `kernel` است، اما چند `interface` سطح بالاتر وجود دارد که برنامه‌ها از طریق آن‌ها با `kernel` تعامل می‌کنند:

- **فایل‌های device در /dev:** برنامه با خواندن و نوشتن روی فایل دستگاه با `driver` ارتباط می‌گیرد.
- **ioctl:** برای ارسال فرمان‌های کنترلی خاص به `driver`ها استفاده می‌شود.
- **procfs و sysfs:** اطلاعات `kernel` و `device`ها را به صورت فایل‌مانند نمایش می‌دهند؛ مثلاً `/proc` و `/sys`.
- **mmap:** برای نگاشت حافظه مشترک یا `memory-mapped I/O` به فضای `user` کاربرد دارد.
- **سوکت netlink:** برای ارتباط ساخت‌یافته بین `user-space` و `kernel` در بخش‌هایی مثل شبکه استفاده می‌شود.
- **سیگنال‌ها:** `kernel` با `signal` می‌تواند رخدادهایی را به پردازنده اطلاع دهد.
- **سازوکار vDSO:** برای بعضی عملیات سبک و پرتکرار، `kernel` کدی را در فضای `user map` می‌کند تا بدون `trap` کامل اجرا شود.

بسیاری از این `interface`ها در نهایت روی `syscall`های پایه مثل `open`، `read`، `write`، `mmap` یا `ioctl` سوار هستند، ولی از دید برنامه‌نویس یک روش ارتباطی جدا و سطح بالاتر محسوب می‌شوند.

پرسش ۳. چرا فقط syscall با DPL_USER فعال است؟

در x86 هر gate در IDT سطح دسترسی دارد. syscall باید از user mode قابل اجرا باشد، چون برنامه user باید بتواند آگاهانه از kernel سرویس بگیرد. بنابراین gate مربوط به T_SYSCALL با DPL_USER تنظیم می‌شود.

اما وقفه‌های سخت‌افزاری و بیشتر exceptionها نباید توسط برنامه user به صورت دلخواه قابل فراخوانی باشند. اگر کاربر می‌توانست هر interrupt gate را با سطح user اجرا کند، می‌توانست مسیرهای حساس kernel را خارج از شرایط واقعی فعال کند و امنیت و پایداری سیستم را از بین ببرد. وقفه سخت‌افزاری باید از سمت سخت‌افزار بیاید و exception نیز باید نتیجه یک رخداد واقعی پردازنده مثل page fault یا divide-by-zero باشد.

پرسش ۴. چرا ss و esp فقط هنگام تغییر سطح دسترسی push می‌شوند؟

وقتی trap باعث تغییر privilege level می‌شود، مثلاً از user mode به kernel mode، پردازنده باید از stack کاربر به stack هسته برود. برای اینکه بعداً بتواند با iret دقیقاً به stack قبلی کاربر برگردد، مقدارهای ss و esp قبلی را روی stack جدید ذخیره می‌کند.

اما اگر trap در همان سطح دسترسی رخ دهد، stack عوض نمی‌شود. پس ss و esp قبلی همان stack جاری هستند و نیازی به ذخیره جداگانه آن‌ها وجود ندارد.

پرسش ۵. نقش argptr و argint

در xv6 آرگومان‌های syscall روی stack سطح کاربر قرار دارند. تابع‌هایی مانند argptr و argint وظیفه دارند این آرگومان‌ها را از فضای user بخوانند و قبل از استفاده در kernel بررسی کنند.

argint یک عدد صحیح را از آرگومان‌های syscall برمی‌دارد. argptr برای pointerها استفاده می‌شود و علاوه بر خواندن آدرس، بررسی می‌کند که بازه مورد نظر داخل address space پردازنده باشد. این بررسی بسیار مهم است، چون اگر kernel به pointer نامعتبر user اعتماد کند، ممکن است:

- از آدرس خارج از حافظه پردازنده بخواند؛
- در آدرس نامعتبر بنویسد و panic رخ دهد؛
- داده kernel یا پردازنده دیگری خراب شود؛
- یک آسیب‌پذیری امنیتی ایجاد شود.

مثلاً در `read(fd, buf, n)` اگر `buf` با `argptr` بررسی نشود، برنامه user می‌تواند آدرسی خارج از حافظه خودش بدهد و kernel هنگام کپی داده در آن آدرس دچار خطا یا خرابی حافظه شود.

بررسی syscall با GDB

برای بررسی مسیر اجرای syscall، برنامه ساده getpidtest.c نوشته شد:

C

```
int main(void)
{
    int pid = getpid();
    printf(1, "getpid() -> %d\n", pid);
    exit();
}
```

برنامه به UPROGS اضافه شد تا داخل xv6 اجرا شود. سپس می‌توان QEMU را در حالت GDB اجرا کرد:

BASH

```
make qemu-gdb
```

و در ترمینال دیگر:

BASH

```
gdb kernel
(gdb) target remote tcp::26000
(gdb) b syscall
```

کاربرد bt

وقتی روی syscall توقف می‌کنیم، دستور زیر زنجیره فراخوانی را نشان می‌دهد:

GDB

```
bt
```

خروجی معمول نشان می‌دهد که مسیر ورود از trap اسمبلی به trap() و سپس به syscall() رسیده است. این backtrace برای فهمیدن این که kernel چگونه از trap handler به dispatcher فراخوانی سیستمی می‌رسد، مفید است.

کاربرد down و بررسی eax

دستور down در GDB در زنجیره frameها به سمت frame پایین‌تر می‌رود. هنگام بررسی syscall، می‌توان frame مربوط به trap را انتخاب کرد و مقدار tf->eax را دید. ثبات eax در wrapper اسمبلی حاوی شماره syscall است. برای getpid مقدار مورد انتظار همان SYS_getpid است که در xv6 برابر ۱۱ است:

GDB

```
p tf->eax
```

ممکن است قبل از رسیدن به اجرای خود `getpidtest` چند `syscall` دیگر مربوط به `shell` یا اجرای برنامه دیده شود. با ادامه دادن اجرا با `c` ، در نهایت توقفی دیده می‌شود که `eax` برابر شماره `getpid` است.

بخش اول عملی: مولد عدد شبه تصادفی

طراحی syscall setSeed

هدف `setSeed` مقداردهی اولیه `state` مولد شبه تصادفی در `kernel` است. طبق صورت پروژه `seed` از ترکیب `tick` فعلی سیستم، `pid` پردازش و بخشی از آدرس `stack` ساخته شد:

C

```
seed = (ticks << 20) ^ (pid << 10) ^ (stack_addr & 0x3FF);
```

چون `ticks` متغیر مشترک `kernel` است، هنگام خواندن آن از `tickslock` استفاده شد. `state` مولد نیز با یک `spinlock` جدا محافظت می‌شود:

C

```
static struct spinlock randlock;  
static uint random_state;
```

این قفل لازم است، چون ممکن است چند پردازش همزمان `syscall random` را صدا بزنند و بدون قفل، مقدار `random_state` ناسازگار شود.

طراحی syscall getRandomNumber

این `syscall` دو آرگومان می‌گیرد: تعداد عددهای مورد نیاز و آدرس بافر `user`. چون صورت پروژه فرض کرده `n < 15` است، در پیاده‌سازی مقدارهای نامعتبر رد می‌شوند:

C

```
if(n <= 0 || n >= 15)  
    return -1;
```

برای بررسی بافر `user` از `argptr` استفاده شد. سپس `state` با رابطه داده‌شده در صورت پروژه به‌روزرسانی می‌شود:

C

```
random_state = (random_state * 1103515245 + 12345) & 0x7fffffff;
```

عددهای تولیدشده با `copyout` در بافر `user` نوشته می‌شوند. استفاده از `copyout` باعث می‌شود نوشتن در حافظه `user` از مسیر کنترل‌شده و وابسته به `page table` همان پردازش انجام شود.

برای تست random یک تاس مجازی نوشته شد. برنامه هم حالت آرگومان خط فرمان را پشتیبانی می‌کند و هم اگر آرگومانی داده نشود، تعداد پرتاب را از ورودی می‌خواند:

BASH

```
$ dice 5
roll 1: 4
roll 2: 2
...
```

یا:

BASH

```
$ dice
number_of_rolls: 5
```

عددهای تولیدشده با رابطه زیر به بازه ۱ تا ۶ تبدیل می‌شوند:

C

```
r = (values[i] % 6) + 1;
```

پرسش‌های بخش random

۱. چرا مولد عدد تصادفی در kernel پیاده‌سازی می‌شود؟

در سیستم‌عامل‌های واقعی، تولید عدد تصادفی امن به entropy نیاز دارد. kernel به رخدادهایی مثل زمان‌بندی وقفه‌ها، ورودی دستگاه‌ها، وضعیت سخت‌افزار و منابع entropy دسترسی دارد. اگر هر برنامه user خودش مستقلاً entropy را مدیریت کند، احتمال تولید خروجی قابل پیش‌بینی و ناامن بیشتر می‌شود. به همین دلیل سیستم‌عامل‌هایی مثل Linux و Windows سرویس مرکزی تولید random دارند و برنامه‌ها از interface‌های رسمی سیستم‌عامل عدد تصادفی می‌گیرند.

۲. اگر دو بار پشت سر هم setSeed اجرا شود، خروجی تغییر می‌کند؟

فرمول seed به ticks، pid و آدرس stack وابسته است. اگر دو فراخوانی پشت سر هم در همان tick، با همان پرده و شرایط مشابه اجرا شوند، ممکن است seed یکسان شود و خروجی تغییر نکند. اما اگر بین دو فراخوانی tick سیستم تغییر کند یا شرایط stack متفاوت شود، seed تغییر می‌کند و دنباله خروجی هم عوض می‌شود. بنابراین تغییر خروجی قطعی نیست، ولی در اجرای معمول احتمال تغییر وجود دارد.

بخش دوم عملی: اطلاعات پردازش

طراحی syscall process_information

این syscall یک pid می‌گیرد و اطلاعات پردازش متناظر را مستقیماً از جدول پردازش‌ها استخراج می‌کند. چون جدول پردازش‌ها ساختار داخلی kernel است، دسترسی به آن باید زیر قفل `ptable.lock` انجام شود.

خروجی مطابق قالب خواسته شده چاپ می‌شود:

TEXT

```
Process_id: 4
Parent_id: 2
Children's_id: 5, 6
Siblings_id: 3, 7
Depth: 2
State: RUNNING
```

برای نمایش state، نام‌های خوانا برای enumهای `procstate` تعریف شد:

C

```
UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE
```

محاسبه parent، children، siblings و depth

- پدر پردازش از فیلد `p->parent` به دست می‌آید.
- فرزندان با پیمایش کل `ptable` و پیدا کردن پردازش‌هایی به دست می‌آیند که `q->parent == p` دارند.
- پردازش‌های هم‌سطح همان پردازش‌هایی هستند که `parent` مشترک دارند و خود پردازش نیستند.
- عمق پردازش با دنبال کردن زنجیره `parent` تا ریشه محاسبه می‌شود.

اگر pid پیدا نشود، syscall مقدار -1 برمی‌گرداند و اگر موفق باشد مقدار 0 می‌دهد.

برنامه تست pinfo_test

برای تست، برنامه‌ای نوشته شد که یک `parent` سه `child` می‌سازد و یکی از `child`ها دو `grandchild` ایجاد می‌کند. با این ساختار، هم `children` و `siblings` دیده می‌شوند و هم `depth` برای سطوح‌های مختلف درخت پردازش قابل بررسی است.

۱. state‌های پردازش در xv6

state	توضیح
UNUSED	خانه جدول پردازش آزاد است و پردازش فعالی ندارد
EMBRYO	پردازش در حال ساخته شدن است ولی هنوز آماده اجرا نیست
SLEEPING	پردازش روی یک channel منتظر رخداد است
RUNNABLE	پردازش آماده اجراست و منتظر CPU است
RUNNING	پردازش روی CPU در حال اجراست
ZOMBIE	پردازش exit کرده ولی parent هنوز wait نکرده است

۲. اگر پردازش pid خودش را بدهد، state چه می‌شود؟

وقتی پردازش syscall را صدا می‌زند، همان پردازش روی CPU در حال اجراست و وارد kernel شده است. بنابراین اگر در همان لحظه اطلاعات خودش را بخواهد، state آن در جدول پردازش‌ها معمولاً `RUNNING` است.

۳. آیا می‌توان `process_information` را در `user-space` پیاده‌سازی کرد؟

در `xv6` خام، `user-space` به `ptable` دسترسی ندارد؛ پس اطلاعات دقیق `parent`، `children`، `siblings` و `state` پردازش‌ها فقط در `kernel` موجود است. بنابراین برای `xv6` این قابلیت باید به شکل `syscall` یا `interface` مشابه در `kernel` پیاده‌سازی شود.

اگر سیستم‌عامل `interface`‌هایی مثل `/proc` داشته باشد، می‌توان ابزار `user-space` نوشت که همان اطلاعات را از فایل‌های مجازی بخواند. مزیت این روش کمتر شدن کد `kernel` و انعطاف بیشتر است. عیب آن این است که نیاز به طراحی `interface` دارد و ممکن است اطلاعات لحظه‌ای به اندازه خواندن مستقیم از `kernel` دقیق نباشد.

بخش سوم عملی: مرتب‌سازی اعداد

طراحی syscall sort_numbers

این syscall نام فایل ورودی را می‌گیرد، اعداد داخل آن را می‌خواند، مرتب می‌کند و خروجی را در فایلی با پسوند `kernel.sorted` می‌نویسد. اگر فایل وجود نداشته باشد یا ورودی نامعتبر باشد، مقدار `-1` برگردانده می‌شود.

برای رعایت محدودیت stack هسته، داده‌های بزرگ روی stack قرار نگرفتند. اعداد با `kalloc` در heap هسته گرفته شدند و فایل با بافر کوچک خوانده شد:

C

```
char buf[128];
nums = (int*)kalloc();
```

خواندن فایل به صورت جریانی انجام می‌شود؛ یعنی لازم نیست کل فایل یک‌باره در یک بافر بزرگ قرار بگیرد. parser فاصله‌ها و `newline`ها را جداکننده عددها در نظر می‌گیرد و عددهای مثبت و منفی را پشتیبانی می‌کند.

کار با فایل در kernel

برای خواندن فایل، ابتدا با `inode namei` پیدا می‌شود و سپس یک `kernel-side struct file` ساخته می‌شود. برای ساخت خروجی نیز از `create` در محدوده فایل‌سیستم استفاده شد. قبل از ساخت فایل خروجی، فایل قبلی با همان نام حذف می‌شود تا خروجی قبلی باقی نماند.

این نکته مهم است چون در `xv6`، ایجاد فایل با `O_CREATE` به معنی `truncate` کردن فایل موجود نیست.

الگوریتم مرتب‌سازی

برای مرتب‌سازی از `insertion sort` غیر بازگشتی استفاده شد. این الگوریتم برای تعداد محدود عددهای پروژه ساده و قابل قبول است و `recursion` ندارد، پس `stack` هسته را درگیر نمی‌کند.

برنامه‌های تست sort

دو برنامه `user-space` نوشته شد:

BASH

```
sort_kernel numbers.txt
sort_user numbers.txt
```

sort_kernel فقط زمان قبل و بعد از syscall را با uptime اندازه می‌گیرد و syscall sort_numbers را صدا می‌زند. sort_user همان کار را در user-space انجام می‌دهد: فایل را با syscalls عادی می‌خواند، در user-space مرتب می‌کند و خروجی را با پسوند user.sorted می‌نویسد.

مقایسه mode switch ها

در مرتب‌سازی kernel، اگر فقط عملیات اصلی sort را حساب کنیم، برنامه user یک بار sort_numbers را صدا می‌زند؛ پس یک ورود از user به kernel و یک برگشت از kernel به user داریم. البته خود برنامه برای اندازه‌گیری زمان uptime را هم صدا می‌زند که دو syscall جداگانه است.

در مرتب‌سازی user-space، تعداد mode switch ها بیشتر است، چون هر عملیات فایل یک syscall جداست:

عملیات	تعداد syscall
ورودی open	۱
ورودی read	به تعداد chunk های خوانده شده
ورودی close	۱
uptime قبل و بعد از sort	۲
خروجی unlink	۱
خروجی open	۱
خروجی write	به تعداد عددها یا خط‌های نوشته شده
خروجی close	۱

هر syscall دو تغییر mode دارد: user به kernel و kernel به user. بنابراین اگر تعداد read ها را R و تعداد write ها را W بگیریم، تعداد تغییر mode در نسخه user-space تقریباً برابر است با:

TEXT

$$2 * (1 + R + 1 + 2 + 1 + 1 + W + 1)$$

نسخه kernel از نظر تعداد ورود و خروج به kernel کمتر است، اما مقدار زیادی منطق application-level را وارد kernel می‌کند.

۱. تعداد تغییر mode ها در هر بخش

در بخش kernel-sort، syscall اصلی `sort_numbers` یک بار اجرا می‌شود؛ یعنی برای خود عملیات اصلی دو تغییر mode داریم. اگر اندازه‌گیری زمان با `uptime` را هم حساب کنیم، دو syscall دیگر اضافه می‌شود و مجموعاً سه syscall و شش تغییر mode خواهیم داشت.

در بخش user-sort، هر `open/read/write/close/unlink/uptime` یک syscall است. بنابراین تعداد تغییر mode ها تابع تعداد chunk های خواندن و تعداد write ها است و معمولاً بسیار بیشتر از kernel-sort می‌شود.

۲. آیا لازم بود sort در kernel پیاده‌سازی شود؟

از نظر طراحی سیستم عامل، خیر. مرتب‌سازی یک کار سطح برنامه است و معمولاً نباید وارد kernel شود. kernel باید سرویس‌های پایه مانند فایل، حافظه، پردازش و I/O را ارائه کند. وارد کردن sort به kernel باعث بزرگ شدن trusted code base، افزایش احتمال crash kernel و کاهش امنیت می‌شود. این بخش بیشتر برای تمرین اضافه کردن syscall و کار با فایل در kernel انجام شده است.

۳. دستور sort در Linux کجاست؟

دستور `sort` در لینوکس یک برنامه user-space است و معمولاً بخشی از GNU coreutils محسوب می‌شود. kernel فقط syscall های لازم برای خواندن فایل، نوشتن فایل، تخصیص حافظه و زمان‌بندی را فراهم می‌کند.

۴. بخش kernel مسئول چه کارهایی است؟

مسئولیت‌های اصلی kernel شامل موارد زیر است:

- مدیریت پردازش‌ها و زمان‌بندی CPU؛
- مدیریت حافظه و حافظه مجازی؛
- کنترل دسترسی و حفاظت بین پردازش‌ها؛
- مدیریت فایل سیستم و device ها؛
- ارائه system call interface؛
- مدیریت وقفه‌ها و exception ها؛
- ارتباط امن بین user-space و سخت‌افزار.

۵. چرا نباید از الگوریتم مرتب‌سازی بازگشتی استفاده کنیم؟

در kernel stack هر پردازش در xv6 بسیار کوچک است. الگوریتم بازگشتی مثل quicksort یا mergesort در بدترین حالت می‌تواند تعداد زیادی frame روی stack بسازد و باعث stack overflow شود. در kernel، stack overflow فقط خطای همان برنامه نیست؛ ممکن است حافظه kernel خراب شود و کل سیستم panic کند. به همین دلیل در این پروژه از مرتب‌سازی غیر بازگشتی استفاده شد.

راهنمای build، اجرا و برنامه‌های اضافه‌شده

برنامه‌های زیر به UPROGS اضافه شدند:

TEXT

```
_dice
_getpidtest
_pinfo_test
_sort_kernel
_sort_user
```

برای build:

BASH

```
make clean
make
make fs.img
```

نمونه دستورهای تست داخل xv6:

BASH

```
dice 10
pinfo_test
sort_kernel numbers.txt
sort_user numbers.txt
getpidtest
```

اعتبارسنجی نهایی و سلامت تحویل

در بازبینی نهایی، تمرکز روی سالم بودن مسیر system call ها، self-contained بودن بسته تحویلی و build مجدد از dist بود. نتیجه نهایی:

مورد بررسی	نتیجه
make clean	موفق
make	موفق
make fs.img	موفق
make dist	موفق
cd dist && make fs.img	موفق
dry-run اجرای CPUS=4	تولید topology صریح برای QEMU با -smp cpus=4, cores=1, threads=1, sockets=4

دو اصلاح تحویلی مهم در بازبینی نهایی اعمال شد:

1. در Makefile، فایل picirq.c و فایل راهنمای پروژه به EXTRA اضافه شد تا خروجی dist مستقل و قابل build بماند.

2. تابع procinfo در proc.c به روش snapshot-then-print اصلاح شد؛ یعنی اطلاعات پردازش زیر ptable.lock جمع‌آوری می‌شود، سپس lock آزاد می‌شود و چاپ با cprintf بعد از آزادسازی lock انجام می‌گیرد. این روش ریسک نگه داشتن lock هنگام چاپ طولانی روی console را حذف می‌کند.

برنامه‌های تست اصلی داخل xv6:

BASH

```
dice 10
pinfo_test
getpidtest
sort_kernel numbers.txt
sort_user numbers.txt
```

در این پروژه مسیر کامل اضافه کردن system call به xv6 از wrapper سطح کاربر تا dispatcher هسته و تابع `sys_*` بررسی شد. `syscalls`های random، اطلاعات پردازش و sort پیاده‌سازی شدند و برای هر کدام برنامه user-space نوشته شد. بخش random اهمیت lock روی state مشترک kernel را نشان داد. بخش process information نشان داد که بعضی اطلاعات فقط از داخل kernel قابل استخراج دقیق هستند. بخش sort نیز از نظر آموزشی مفید بود، اما از نظر طراحی نشان داد که وارد کردن منطق برنامه‌ای به kernel همیشه انتخاب خوبی نیست و باید با احتیاط انجام شود.